

StudyBuddy

Deployment Readiness Report

Pre-Production Analysis for Online Tutoring Marketplace

Report Date: May 26, 2026

Technology Stack: MongoDB · Express 5 · React 19 · Vite 6 · Socket.io

Document Type: Pre-Production Deployment Analysis

Executive Summary

StudyBuddy is a feature-complete MVP for an online tutoring marketplace supporting students, tutors, and administrators. The platform includes tutor discovery, session booking, mock checkout with a 10% platform fee, real-time notifications, and admin governance tools.

Verdict: Suitable for demo or staging deployment after critical security fixes. Not ready for real-money production without live payment integration, security hardening, and production infrastructure.

1. Readiness Overview

Area	Status	Notes
Core features	' Ready	Booking !' payment !' confirmation flow works
UI/UX	' Good	Professional checkout, dashboards, admin panel
Authentication	& p Partial	JWT works; role registration needs hardening
Payments	'L Mock only	Test cards only; no live payment gateway
Security	& p Needs work	CORS, secrets, rate limiting gaps
DevOps	'L Missing	No Docker, CI/CD, or deployment docs
Documentation	'L Missing	README is default Vite template

2. Project Architecture

StudyBuddy follows a classic MERN-style split architecture:

- Frontend: React 19 + Vite 6 (port 5173 in development)
- Backend: Express 5 + Socket.io (port 5003 in local configuration)
- Database: MongoDB (study database)
- API prefix: /api — auth, tutors, bookings, payments, reviews, students, admin
- Real-time: Socket.io events for booking confirmed, cancelled, and completed

Key frontend routes include public pages (/login, /register), student flows (/tutors, /booking/:tutorId), tutor dashboard, and admin management (/dashboard/admin, /admin/users, /admin/tutors).

3. Feature Inventory

Module	Status	Details
Authentication	Complete	Register, login, JWT, bcrypt password hashing
Student flow	Complete	Search, profile, booking, payment, dashboard
Tutor flow	Complete	Profile setup, approval, sessions, earnings
Admin flow	Complete	Users, tutors, approvals, audit logs, 10% profit
Booking system	Complete	Slots, conflict checks, status lifecycle
Payments	Mock only	Test cards; no real payment processor
Platform fee (10%)	Complete	Calculated server-side; shown at checkout
Reviews	Complete	Post-session student reviews
Meeting links	Simulated	Mock Zoom URLs generated on payment

4. Critical Blockers

4.1 Payments Are Mock-Only

The payment controller uses hardcoded test card numbers (4111111111111111 for success, 4000000000000002 for decline). The @paypal/checkout-server-sdk package is installed but not integrated. Real money cannot be collected in production.

Action: Integrate Stripe or PayPal before accepting live payments. Retain mock mode for development only.

4.2 Environment Secrets Not Protected

.gitignore does not exclude .env or backend/.env files. JWT secrets and database URIs could be committed to version control.

Action: Add .env files to .gitignore, rotate JWT_SECRET if ever committed, and provide .env.example templates.

4.3 CORS Fully Open on REST API

Express uses app.use(cors()) without origin restriction, while Socket.io correctly limits origins to CLIENT_URL. Any website can call the REST API in production.

Action: Restrict CORS to process.env.CLIENT_URL with credentials enabled.

4.4 Admin Role Registration Loophole

The registration UI only offers student and tutor roles, but POST /api/auth/register accepts any role from the request body, including admin.

Action: Whitelist roles on the server to student and tutor only. Create admin accounts manually or via a secure seed script.

4.5 No Production Hosting Configuration

Missing Dockerfile, docker-compose, CI/CD pipeline, deployment README, health check endpoint, process manager configuration, and reverse proxy setup for SPA routing.

5. High-Priority Issues

Port configuration inconsistency

Location	Default Port
backend/server.js	5000
vite.config.js proxy	5003
src/services/api.js	5003
SocketContext.jsx fallback	5000

Action: Standardize PORT, VITE_API_URL, VITE_SOCKET_URL, and CLIENT_URL across all environments.

MongoDB is local-only

Current MONGODB_URI points to mongodb://localhost:27017/study. Production requires MongoDB Atlas or another managed database with IP allowlisting and strong credentials.

SPA routing in production

BrowserRouter requires the host to serve index.html for all client-side routes. Configure fallback routing on Vercel, Netlify, or Nginx.

Admin time-range filter non-functional

Admin dashboard sends ?range=week|month|year but getPlatformStats ignores the parameter and always returns the last 7 days.

Tutor approval role assignment

approveTutor sets isApproved but may not set User.role to tutor, causing booking failures for approved tutors.

Avatar upload incomplete

tutorController references req.file and /uploads/ but multer middleware and static file serving are not configured.

6. Security Assessment

Control	Status
Password hashing (bcrypt)	Implemented
JWT on protected routes	Implemented
RBAC (student/tutor/admin)	Partial — register loophole
Payment authorization	Implemented
Audit logs	Implemented
Rate limiting	Not implemented
Helmet (HTTP headers)	Not implemented
Input validation library	Manual checks only
HTTPS enforcement	Deployment responsibility
Secrets in environment variables	Partial — not gitignored

7. Environment Variables

Backend (backend/.env)

Variable	Required	Notes
MONGODB_URI	Yes	MongoDB Atlas connection string in production
JWT_SECRET	Yes	Long random string (32+ characters)
JWT_EXPIRE	Yes	e.g. 7d or 30d
PORT	Yes	Host-assigned or fixed e.g. 5003
CLIENT_URL	Yes	Production frontend URL for CORS/Socket.io
NODE_ENV	Recommended	Set to production

Frontend (root .env — build-time)

Variable	Required	Example
VITE_API_URL	Yes	https://api.yourdomain.com/api
VITE_SOCKET_URL	Yes	https://api.yourdomain.com

Note: Vite embeds VITE_* variables at build time. Rebuild the frontend when these values change.

8. Recommended Deployment Architecture

Option A — Split Hosting (Recommended)

Component	Service	Notes
Frontend	Vercel / Netlify / Cloudflare Pages	npm run build !' serve dist/
Backend	Render / Railway / Fly.io	npm start in backend/
Database	MongoDB Atlas	M0 free tier suitable for staging
Domain & SSL	Cloudflare	DNS + HTTPS

Option B — Single VPS

- Nginx: SSL termination, /api !' Node, / !' static dist/, /socket.io !' WebSocket proxy
- PM2 for Node process management
- MongoDB Atlas (avoid self-hosting DB initially)

9. Pre-Deployment Checklist

Must Do

- ☐ Fix admin registration API role whitelist
- ☐ Add .env to .gitignore and rotate secrets
- ☐ Restrict CORS to CLIENT_URL
- ☐ Set up MongoDB Atlas
- ☐ Configure all production environment variables
- ☐ Run npm run build and verify success
- ☐ Configure SPA fallback routing
- ☐ Align PORT, VITE_API_URL, VITE_SOCKET_URL, CLIENT_URL
- ☐ Replace mock payments or label site as demo-only

Should Do

- ☐ Add helmet and express-rate-limit
- ☐ Fix tutor approval role assignment
- ☐ Add GET /api/health endpoint
- ☐ Write deployment README and .env.example files
- ☐ Remove debug console.log from production

- ☐ Fix or remove admin time-range filter UI

10. Build & Deploy Commands

Frontend: `npm install && npm run build` (outputs to `dist/`)

Backend: `cd backend && npm install && npm start`

Preview: `npm run preview` (test production frontend build locally)

11. Post-Deployment Verification

- ☐ Register as student and tutor
- ☐ Admin approves tutor application
- ☐ Student searches, books, and completes checkout
- ☐ Checkout shows 10% platform fee and correct total
- ☐ Payment succeeds and booking is confirmed
- ☐ Admin dashboard reflects updated platform profit
- ☐ Tutor receives booking; student gets meeting link
- ☐ Socket notifications fire on booking events
- ☐ Direct URL access works for all routes
- ☐ 401 responses redirect to login when token expires

12. Revenue Model Summary

Item	Calculation	Example (\$50/hr × 1hr)
Session subtotal	hourly rate × duration	\$50.00
Platform fee	10% of subtotal	\$5.00
Student pays	subtotal + platform fee	\$55.00
Admin profit	sum of platform fees (paid bookings)	\$5.00
Tutor earnings	session subtotal (completed sessions)	\$50.00

13. Final Recommendation

Deployment Type	Ready?
University demo / portfolio	Yes — after critical security fixes (Section 4)
Beta with test users (no real money)	Yes — with mock payments clearly labeled
Production with real payments	No — requires live gateway, security hardening, monitoring

Minimum Path to Staging (1–2 days)

- MongoDB Atlas + production environment variables
- Security fixes: CORS, role whitelist, `.gitignore`
- Deploy frontend and backend to separate hosts

- End-to-end testing on staging URL

Minimum Path to Production (1–2 weeks)

- All staging requirements above
- Stripe or PayPal live integration
- Rate limiting, Helmet, structured logging, error monitoring
- Deployment documentation and MongoDB backup strategy

Generated automatically on May 26, 2026 · StudyBuddy Deployment Readiness Report · Confidential